

My Leetcode

27 solved problems • automatically generated

Each problem gets one page with a large, readable solution code section.

1. Two Sum

Difficulty: Easy | **Pattern:** Hashing / Set

Question

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

Example

Input: `nums = [2,7,11,15]`, `target = 9` Output: `[0,1]` Explanation: Because `nums[0] + nums[1] == 9`

Short Answer / Approach

Use the repository solution below. The key pattern is Hashing / Set.

Complexity

Time: $O(n)$ average | Space: $O(n)$

C++ Solution

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> mp;
        for (int i = 0; i < nums.size(); i++) {
            int x = target - nums[i];
            if (mp.find(x) != mp.end()) {
                return {mp[x], i};
            }
            mp[nums[i]] = i;
        }
        return {};
    }
};
```

Problem 1 of 27

2. Add Two Numbers

Difficulty: Medium | **Pattern: Linked List**

Question

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

Example

Input: l1 = [2,4,3], l2 = [5,6,4] Output: [7,0,8] Explanation: 342 + 465 = 807.

Short Answer / Approach

Use the repository solution below. The key pattern is Linked List.

Complexity

Time: O(n) | Space: O(1) auxiliary

C++ Solution

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        ListNode* dummy = new ListNode(0);
        ListNode* temp = dummy;
        int carry = 0;
        while (l1 != NULL || l2 != NULL || carry != 0) {
            int sum = carry;
            if (l1 != NULL) {
                sum += l1->val;
                l1 = l1->next;
            }
            if (l2 != NULL) {
                sum += l2->val;
                l2 = l2->next;
            }
            carry = sum / 10;
            temp->next = new ListNode(sum % 10);
            temp = temp->next;
        }
        return dummy->next;
    }
};
```

Problem 2 of 27

3. Longest Substring Without Repeating Characters

Difficulty: Medium | **Pattern: Hashing / Set**

Question

Given a string *s*, find the length of the longest substring without duplicate characters.

Example

Input: *s* = "abcabcbb" Output: 3 Explanation: The answer is "abc", with the

Short Answer / Approach

Use the repository solution below. The key pattern is Hashing / Set.

Complexity

Time: $O(n)$ average | Space: $O(n)$

C++ Solution

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        unordered_set<char> st;
        int left = 0;
        int maxi = 0;
        for (int right = 0; right < s.size(); right++) {
            while(st.find(s[right]) != st.end()) {
                st.erase(s[left]);
                left++;
            }
            st.insert(s[right]);
            maxi = max(maxi, right - left + 1);
        }
        return maxi;
    }
};
```

Problem 3 of 27

4. Median of Two Sorted Arrays

Difficulty: Hard | **Pattern:** Sorting

Question

Given two sorted arrays nums1 and nums2 of size m and n respectively, return the median of the two sorted arrays.

Example

Input: nums1 = [1,3], nums2 = [2] Output: 2.00000 Explanation: merged array = [1,2,3] and median = 2.00000

Short Answer / Approach

Use the repository solution below. The key pattern is Sorting.

Complexity

Time: $O(n \log n)$ typical | Space: $O(n)$ or $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
        nums1.insert(nums1.end(), nums2.begin(), nums2.end());
        sort(nums1.begin(), nums1.end());
        if(nums1.size()%2==0){
            return (nums1[(nums1.size()/2)-1]+nums1[nums1.size()/2])/2.0;
        }
        else{
            return nums1[nums1.size()/2.0];
        }
        return{};
    }
};
```

Problem 4 of 27

7. Reverse Integer

Difficulty: Medium | **Pattern: Math / Simulation**

Question

Given a signed 32-bit integer x , return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, then return 0.

Example

Input: $x = 123$ Output: 321

Short Answer / Approach

Use the repository solution below. The key pattern is Math / Simulation.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    int reverse(int x) {
        int rev = 0;

        while(x!=0){
            int d = x%10;
            if(rev > INT_MAX/10 || rev < INT_MIN/10)
                return 0;
            rev = rev*10 + d;
            x=x/10;
        }
        return rev;
    }
};
```

Problem 5 of 27

9. Palindrome Number

Difficulty: Easy | **Pattern:** Math / Simulation

Question

Given an integer x, return true if x is a palindrome, and false otherwise.

Example

Input: x = 121 Output: true Explanation: 121 reads as 121 from left to right and from right to

Short Answer / Approach

Use the repository solution below. The key pattern is Math / Simulation.

Complexity

Time: O(n) | Space: O(1) auxiliary

C++ Solution

```
class Solution {
public:
    bool isPalindrome(int x){
        if (x < 0)
            return false;
        if (x != 0 && x % 10 == 0)
            return false;
        int rev = 0;
        while (x > rev) {
            rev = rev * 10 + x % 10;
            x /= 10;
        }
        if(x==rev || x==rev/10)
            return true;
        else
            return false;
    }
};
```

Problem 6 of 27

11. Container With Most Water

Difficulty: Medium | **Pattern: Two Pointers**

Question

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are `(i, 0)` and `(i, height[i])`.

Example

Input: `height = [1,8,6,2,5,4,8,3,7]` Output: 49 Explanation: The above vertical lines are represented by the array `height`.

Short Answer / Approach

Use the repository solution below. The key pattern is Two Pointers.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    int maxArea(vector<int>& height) {
        int left = 0;
        int right = height.size()-1;
        int area = 0;
        while(left<right){
            int h = min(height[left],height[right]);
            int a = h*(right-left);
            area = max(a,area);
            if(height[left]<height[right]){
                left++;
            }
            else{
                right--;
            }
        }
        return area;
    }
};
```

Problem 7 of 27

14. Longest Common Prefix

Difficulty: Easy | **Pattern:** Arrays / Implementation

Question

Write a function to find the longest common prefix string amongst an array of strings.

Example

Input: strs = ["flower","flow","flight"] Output: "fl"

Short Answer / Approach

Use the repository solution below. The key pattern is Arrays / Implementation.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    string longestCommonPrefix(vector<string>& strs) {
        string prefix = strs[0];
        for(int i = 1; i < strs.size(); i++) {
            int j = 0;
            while(j < prefix.size() &&
                j < strs[i].size() &&
                prefix[j] == strs[i][j]) {
                j++;
            }
            prefix = prefix.substr(0, j);
            if(prefix == "")
                return "";
        }
        return prefix;
    }
};
```

Problem 8 of 27

19. Remove Nth Node From End of List

Difficulty: Medium | **Pattern:** Linked List

Question

Given the head of a linked list, remove the nth node from the end of the list and return its head.

Example

Input: head = [1,2,3,4,5], n = 2 Output: [1,2,3,5]

Short Answer / Approach

Use the repository solution below. The key pattern is Linked List.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode *dummy = new ListNode(0);
        dummy->next = head;
        ListNode *temp=head;
        int count = 0;
        while(temp!=NULL){
            temp = temp->next;
            count++;
        }
        int k=count-n;
        temp = dummy;
        while(k>0){
            temp=temp->next;
            k--;
        }
        ListNode *templ=temp->next;
        temp->next=templ->next;
        delete templ;
        ListNode* newHead = dummy->next;
        delete dummy;
        return newHead;
    }
};
```

Problem 9 of 27

21. Merge Two Sorted Lists

Difficulty: Easy | **Pattern:** Linked List

Question

You are given the heads of two sorted linked lists list1 and list2.

Example

Input: list1 = [1,2,4], list2 = [1,3,4] Output: [1,1,2,3,4,4]

Short Answer / Approach

Use the repository solution below. The key pattern is Linked List.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        if(list1==NULL){return list2;}
        if(list2==NULL){return list1;}
        ListNode* dummy=new ListNode(-1);
        ListNode* temp = dummy;
        while(list1!=NULL && list2!=NULL){
            if(list1->val<=list2->val){
                temp->next=list1;
                list1=list1->next;
            }
            else{
                temp->next=list2;
                list2=list2->next;
            }
            temp=temp->next;
        }
        if(list1!=NULL){
            temp->next=list1;
        }
        else{
            temp->next=list2;
        }
        return dummy->next;
    }
};
```

Problem 10 of 27

24. Swap Nodes in Pairs

Difficulty: Medium | **Pattern: Linked List**

Question

Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Short Answer / Approach

Use the repository solution below. The key pattern is Linked List.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        ListNode* dummy = new ListNode(0);
        dummy->next = head;
        ListNode* temp = dummy;
        while(temp->next != NULL && temp->next->next != NULL) {
            ListNode* first = temp->next;
            ListNode* second = first->next;
            first->next = second->next;
            second->next = first;
            temp->next = second;
            temp = first;
        }
        ListNode* newHead = dummy->next;
        delete dummy;
        return newHead;
    }
};
```

Problem 11 of 27

33. Search in Rotated Sorted Array

Difficulty: Medium | **Pattern: Arrays / Implementation**

Question

There is an integer array `nums` sorted in ascending order (with distinct values).

Example

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0` Output: `4`

Short Answer / Approach

Use the repository solution below. The key pattern is Arrays / Implementation.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        for(int i=0;i<nums.size();i++){
            if(nums[i]==target){
                return i;
            }
        }
        return -1;
    }
};
```

Problem 12 of 27

34. Find First and Last Position of Element in Sorted Array

Difficulty: Medium | **Pattern: Binary Search**

Question

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value.

Example

Input: `nums = [5,7,7,8,8,10]`, `target = 8` Output: `[3,4]`

Short Answer / Approach

Use the repository solution below. The key pattern is Binary Search.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        int first = -1;
        int last = -1;
        // Find first occurrence
        int left = 0;
        int right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                first = mid;
                right = mid - 1; // search left
            }
            else if (target > nums[mid]) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }
        // Find last occurrence
        left = 0;
        right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                last = mid;
                left = mid + 1; // search right
            }
            else if (target > nums[mid]) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }
        return {first, last};
    }
};
```

Problem 13 of 27

35. Search Insert Position

Difficulty: Easy | **Pattern:** Binary Search

Question

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

Example

Input: nums = [1,3,5,6], target = 5 Output: 2

Short Answer / Approach

Use the repository solution below. The key pattern is Binary Search.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size()-1;
        while(left<=right){
            int mid = left + (right - left)/2;
            if(nums[mid]==target){
                return mid;
            }
            if(target>nums[mid]){
                left=mid+1;
            }
            else{
                right=mid-1;
            }
        }
        return left;
    }
};
```

Problem 14 of 27

49. Group Anagrams

Difficulty: Medium | **Pattern: Hashing / Set**

Question

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

Short Answer / Approach

Use the repository solution below. The key pattern is Hashing / Set.

Complexity

Time: $O(n \log n)$ typical | Space: $O(n)$ or $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    vector<vector<string>>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>>> mp;
        for(string s:strs){
            string key = s;
            sort(key.begin(),key.end());
            mp[key].push_back(s);
        }
        vector<vector<string>>> ans;
        for(auto x:mp){
            ans.push_back(x.second);
        }
        return ans;
    }
};
```

Problem 15 of 27

53. Maximum Subarray

Difficulty: Medium | **Pattern: Greedy / Kadane**

Question

Given an integer array `nums`, find the subarray with the largest sum, and return its sum.

Example

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]` Output: 6 Explanation: The subarray `[4,-1,2,1]` has the largest sum.

Short Answer / Approach

Use the repository solution below. The key pattern is Greedy / Kadane.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int sum = 0;
        int maxsum = INT_MIN;
        for(int i = 0; i<nums.size(); i++){
            sum+=nums[i];
            maxsum = max(maxsum, sum);
            if(sum<0){
                sum=0;
            }
        }
        return maxsum;
    }
};
```

Problem 16 of 27

76. Minimum Window Substring

Difficulty: Hard | **Pattern:** Hashing / Set

Question

Given two strings *s* and *t* of lengths *m* and *n* respectively, return the minimum window substring of *s* such that every character in *t* (including duplicates) is included in the window. If there is no such substring, return the empty string "".

Example

Input: *s* = "ADOBECODEBANC", *t* = "ABC" Output: "BANC" Explanation:

Short Answer / Approach

Use the repository solution below. The key pattern is Hashing / Set.

Complexity

Time: $O(n)$ average | Space: $O(n)$

C++ Solution

```
class Solution {
public:
    string minWindow(string s, string t) {
        unordered_map<char,int>n,w;
        for(char c:t) n[c]++;
        int l = 0,start=0;
        int req = n.size();
        int have = 0;
        int len = INT_MAX ;
        for(int r = 0; r<s.size(); r++){
            w[s[r]]++;
            if(n.count(s[r]) && w[s[r]]==n[s[r]]){
                have++;
            }
            while(have == req){
                if(r-l+1 < len){
                    len = r-l+1;
                    start = l;
                }
                w[s[l]]--;
                if(n.count(s[l])&&w[s[l]]<n[s[l]]){
                    have--;
                }
                l++;
            }
        }
        if(len == INT_MAX) return "";
        return s.substr(start,len);
    }
};
```

Problem 17 of 27

81. Search in Rotated Sorted Array II

Difficulty: Medium | **Pattern: Binary Search**

Question

There is an integer array `nums` sorted in non-decreasing order (not necessarily with distinct values).

Example

Input: `nums = [2,5,6,0,0,1,2]`, `target = 0` Output: `true`

Short Answer / Approach

Use the repository solution below. The key pattern is Binary Search.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    bool search(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target)
                return true;
            // Duplicates: cannot determine which half is sorted
            if (nums[left] == nums[mid] && nums[mid] == nums[right]) {
                left++;
                right--;
            }
            // Left half is sorted
            else if (nums[left] <= nums[mid]) {
                if (nums[left] <= target && target <= nums[mid])
                    right = mid - 1;
                else
                    left = mid + 1;
            }
            // Right half is sorted
            else {
                if (nums[mid] < target && target <= nums[right])
                    left = mid + 1;
                else
                    right = mid - 1;
            }
        }
        return false;
    }
};
```

Problem 18 of 27

94. Binary Tree Inorder Traversal

Difficulty: Easy | **Pattern:** Binary Search

Question

Given the root of a binary tree, return the inorder traversal of its nodes' values.

Short Answer / Approach

Use the repository solution below. The key pattern is Binary Search.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    vector<int> ans;
    vector<int> inorderTraversal(TreeNode* root) {
        if(root == NULL)
            return ans;
        inorderTraversal(root->left);
        ans.push_back(root->val);
        inorderTraversal(root->right);
        return ans;
    }
};
```

Problem 19 of 27

121. Best Time to Buy and Sell Stock

Difficulty: Easy | **Pattern:** Greedy / Kadane

Question

You are given an array prices where prices[i] is the price of a given stock on the ith day.

Example

Input: prices = [7,1,5,3,6,4] Output: 5 Explanation: Buy on day 2 (price = 1) and sell on day 5

Short Answer / Approach

Use the repository solution below. The key pattern is Greedy / Kadane.

Complexity

Time: O(n) | Space: O(1) auxiliary

C++ Solution

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int minprice = prices[0];
        int profit = 0;
        for(int i =1;i<prices.size();i++){
            minprice= min(minprice,prices[i]);
            profit = max(profit,prices[i]-minprice);
        }
        return profit;
    }
};
```

Problem 20 of 27

122. Best Time to Buy and Sell Stock II

Difficulty: Medium | **Pattern:** Greedy / Kadane

Question

You are given an integer array `prices` where `prices[i]` is the price of a given stock on the `i`th day.

Example

Input: `prices = [7,1,5,3,6,4]` Output: 7 Explanation: Buy on day 2 (price = 1) and sell on day 3

Short Answer / Approach

Use the repository solution below. The key pattern is Greedy / Kadane.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int profit=0;
        for(int i = 1;i<prices.size();i++){
            if(prices[i]>prices[i-1]){
                profit = profit + prices[i] - prices[i-1];
            }
        }
        return profit;
    }
};
```

Problem 21 of 27

128. Longest Consecutive Sequence

Difficulty: Medium | **Pattern:** Hashing / Set

Question

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

Example

Input: `nums = [100,4,200,1,3,2]` Output: 4 Explanation: The longest consecutive elements sequence is [1,2,3,4].

Short Answer / Approach

Use the repository solution below. The key pattern is Hashing / Set.

Complexity

Time: $O(n)$ average | Space: $O(n)$

C++ Solution

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> st(nums.begin(),nums.end());
        int ans =0;
        for(int x:st){
            if(st.find(x-1)==st.end()){
                int current =x;
                int count =1;
                while(st.find(current+1)!=st.end()){
                    current++;
                    count++;
                }
                ans=max(ans,count);
            }
        }
        return ans;
    }
};
```

Problem 22 of 27

189. Rotate Array

Difficulty: Medium | **Pattern: Math / Simulation**

Question

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example

Input: `nums = [1,2,3,4,5,6,7]`, `k = 3` Output: `[5,6,7,1,2,3,4]` Explanation: rotate 1 steps to the

Short Answer / Approach

Use the repository solution below. The key pattern is Math / Simulation.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    void rotate(vector<int>& nums, int k) {
        k = k % nums.size();
        reverse(nums.begin(), nums.end());
        reverse(nums.begin(), nums.begin() + k);
        reverse(nums.begin() + k, nums.end());
    }
};
```

Problem 23 of 27

209. Minimum Size Subarray Sum

Difficulty: Medium | **Pattern:** Arrays / Implementation

Question

Given an array of positive integers `nums` and a positive integer `target`, return the minimal length of a subarray whose sum is greater than or equal to `target`. If there is no such subarray, return 0 instead.

Example

Input: `target = 7`, `nums = [2,3,1,2,4,3]` Output: 2 Explanation: The subarray `[4,3]` has the minimal length.

Short Answer / Approach

Use the repository solution below. The key pattern is Arrays / Implementation.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int l=0;
        int sum=0;
        int len = INT_MAX;
        for(int r=0;r<nums.size();++r){
            sum += nums[r];
            while(sum>=target){
                len = min(r-l+1,len);
                sum-=nums[l];
                ++l;
            }
        }
        if(len == INT_MAX) return 0;
        return len;
    }
};
```

Problem 24 of 27

215. Kth Largest Element in an Array

Difficulty: Medium | **Pattern:** Arrays / Implementation

Question

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array.

Example

Input: `nums = [3,2,1,5,6,4]`, `k = 2` Output: 5

Short Answer / Approach

Use the repository solution below. The key pattern is Arrays / Implementation.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        priority_queue<int, vector<int>, greater<int>> pq;
        for(int x:nums){
            pq.push(x);
            if(pq.size()>k){
                pq.pop();
            }
        }
        return pq.top();
    }
};
```

Problem 25 of 27

239. Sliding Window Maximum

Difficulty: Hard | **Pattern:** Two Pointers

Question

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Example

Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3` Output: `[3,3,5,5,6,7]` Explanation: Window position Max

Short Answer / Approach

Use the repository solution below. The key pattern is Two Pointers.

Complexity

Time: $O(n)$ | Space: $O(1)$ auxiliary

C++ Solution

```
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        deque<int> dq;
        vector<int> ans;
        for (int r = 0; r < nums.size(); r++) {
            // Remove elements outside the window
            while (!dq.empty() && dq.front() <= r - k) {
                dq.pop_front();
            }
            // Remove smaller elements
            while (!dq.empty() && nums[dq.back()] <= nums[r]) {
                dq.pop_back();
            }
            // Add current index
            dq.push_back(r);
            // Window size is k
            if (r >= k - 1) {
                ans.push_back(nums[dq.front()]);
            }
        }
        return ans;
    }
};
```

Problem 26 of 27

4080. Smallest Missing Multiple of K

Difficulty: Easy | **Pattern:** Hashing / Set

Question

Given an integer array `nums` and an integer `k`, return the smallest positive multiple of `k` that is missing from `nums`.

Short Answer / Approach

Use the repository solution below. The key pattern is Hashing / Set.

Complexity

Time: $O(n)$ average | Space: $O(n)$

C++ Solution

```
class Solution {
public:
    int missingMultiple(vector<int>& nums, int k) {
        unordered_set<int> st;
        for(int x : nums){
            st.insert(x);
        }
        for(int i=1;i<=nums.size()+1;i++){
            int x = k*i;
            if(st.find(x)==st.end()){
                return x;
            }
        }
        return -1;
    }
};
```

Problem 27 of 27